

Data Link Layer

The concerns at the Data Link Layer include:

- 1. What services should be provided to upper layers?**
- 2. Framing,**
- 3. Error Control.**
- 4. Flow Control.**

Goals of the data link layer

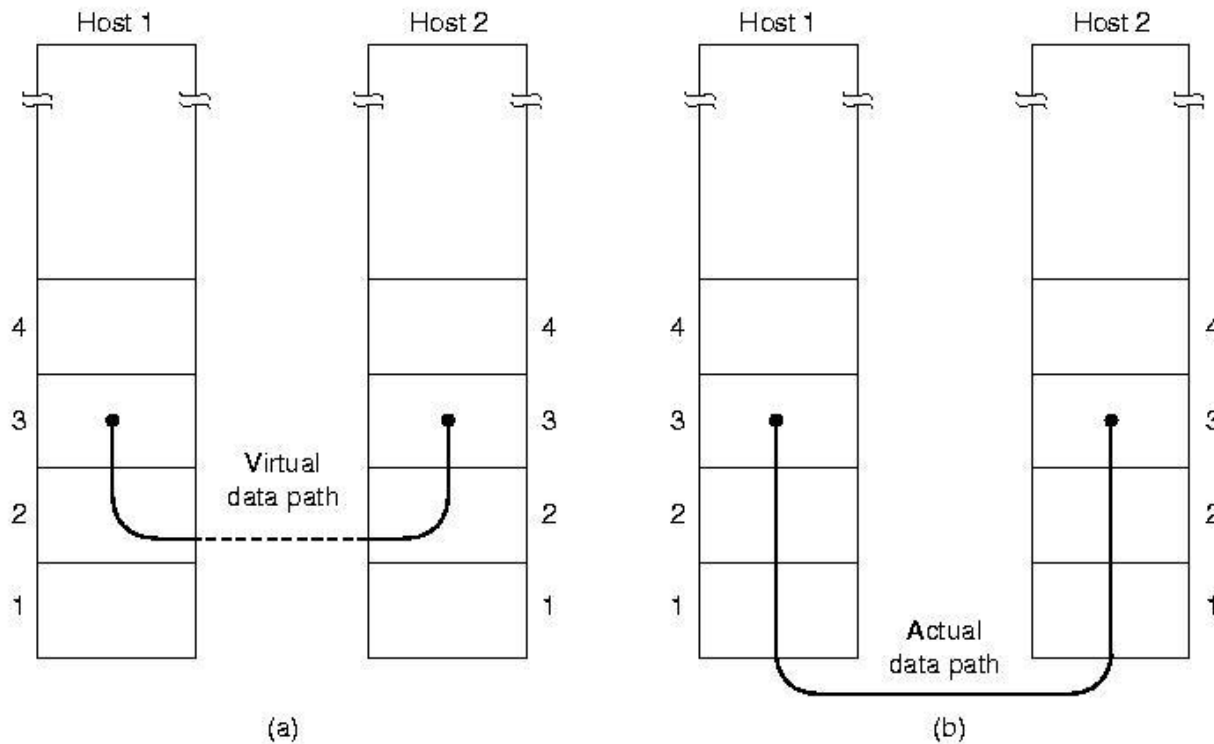
The goal of the data link layer is to provide reliable, efficient communication between adjacent machines connected by a single communication channel. Specifically:

1. Group the physical layer bit stream into units called **frames**.
2. Sender **checksums** the frame and transmits checksum together with data.
3. Receiver re-computes the **checksum** and compares it with the received value. If they differ, an error has occurred and the frame is discarded.
4. Perhaps return a positive or negative **acknowledgment** to the sender. A positive acknowledgment indicate the frame was received without errors, while a negative acknowledgment indicates the opposite.
5. **Flow control**. Prevent a fast sender from overpowering a slower receiver. For example, a supercomputer can easily generate data faster than a PC can consume it.
6. In general, provide **service** to the **network** layer.

1. SERVICES PROVIDED TO THE NETWORK LAYER

There are several possible kinds of services that can be provided to network layers.

The Figure is a reminder of the difference between virtual and actual communications between layers.



Three types of services provided to network layer:

- **Unacknowledged Connection-less Service -- Best Effort**
- **Acknowledged Connection-less Service -- Acknowledged Delivery**
- **Acknowledged Connection-Oriented Service -- Reliable Delivery**

Unacknowledged Connection-less Service -- Best Effort:

The receiver does not return acknowledgments to the sender, so the sender has no way of knowing if a frame has been successfully delivered.

When would such a service be appropriate?

1. When higher layers can recover from errors with little loss in performance.
2. For real-time applications requiring "better never than late" semantics. Old data may be worse than no data.

Acknowledged Connection-less Service -- Acknowledged Delivery:

- The receiver returns an acknowledgment frame to the sender indicating that a data frame was properly received. The sender keeps connection state, but may not necessarily retransmit unacknowledged frames.
- Likewise, the receiver may collect the received frames to higher layers in the order in which they arrive, regardless of the original sending order.
- Typically, each frame is assigned a unique sequence number, which the receiver returns in an acknowledgment frame to indicate which frame the ACK refers to. The sender must retransmit unacknowledged (e.g., lost or damaged) frames.

Acknowledged Connection-Oriented Service -- Reliable Delivery:

- Frames are delivered to the receiver reliably and in the same order as generated by the sender.
- Connection state keeps track of sending order and which frames require retransmission. For example, receiver state includes which frames have been received, which ones have not, etc.

2. FRAMING

The DLL translates the physical layer's raw bit stream into discrete units (messages) called frames.

How can frame be transmitted so the receiver can detect frame boundaries? That is, how can the receiver recognize the start and end of a frame?

We will discuss four ways of framing:

Character Count

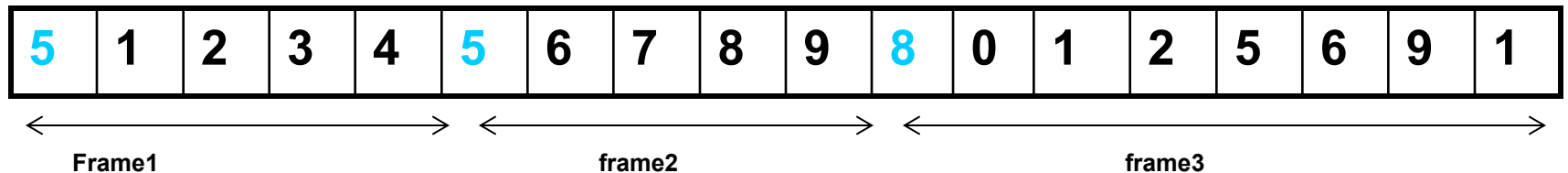
Flag Byte with Byte Stuffing

Starting and Ending Flag with Bit Stuffing

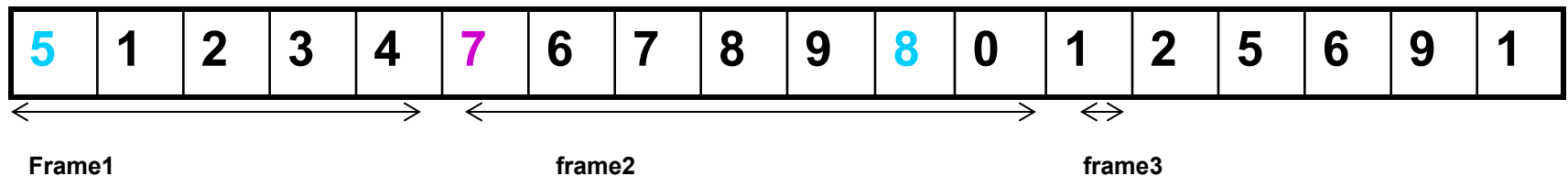
Encoding Violations

Character Count:

- Make the first field in the frame's header be the length of the frame. That way the receiver knows how big the current frame is and can determine where the next frame ends.
- **Disadvantage:** Receiver loses synchronization when bits become garbled. If the bits in the count become corrupted during transmission, the receiver will think that the frame contains fewer (or more) bits than it actually does.
- Although checksum will detect the frames are incorrect, the receiver will have difficulty re-synchronizing to the start of a new frame. This technique is not used anymore, since better techniques are available.



Without error



With an error

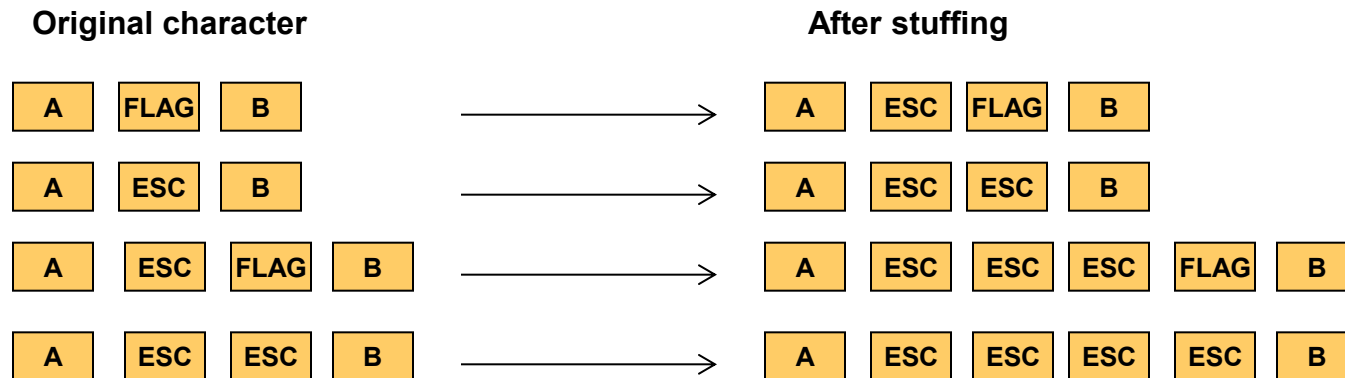
Flag byte with byte stuffing

Use reserved character to indicate the start and end of a frame. For instance, use a character FLAG to signal the beginning and ending of a frame.



(a) A frame delimited by flag byte

What happens if the FLAG byte appear in the frame itself?



(b) Byte sequences before and after stuffing

Disadvantage: use of 8-bit character. Not all character code uses 8-bit character. E.g. Unicode uses 16-bit char.

Starting and Ending Flag with Bit Stuffing:

Each frame uses reserved bit patterns **01111110** to indicate the start and end of a frame. Whenever the sender data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit after five 1s into the outgoing bit stream.

Whenever the receiver sees the five consecutive incoming 1 bits, followed by a 0 bit, it automatically delete the 0 bit.

(a) 011011111 11111 11111 0010

(b) 01111110 0110111110111110111110 0010 01111110

(c) 011011111 11111 11111 0010

(a) The original data (b) the data as they appears on the line (c) the data as they are stored in the receiver's memory after destuffing.

Encoding Violations:

Encoding uses at physical medium. In Manchester encoding, for instance, 1-bits are represented by a **– to + voltage**, and 0-bits by **+ to - voltage**. The start/end of a frame could be represented by the signal **low-low** or **high-high**.

The advantage of encoding violations is that no extra bandwidth is required as in bit or character stuffing.

3. ERROR CONTROL

Must insure that all frames are delivered (possibly in order) to a destination. Three components are required to do this:

Acknowledgments, Timers, and Sequence Numbers

Acknowledgments:

- Reliable delivery is achieved using the "acknowledgments with retransmission" paradigm.
- The receiver returns a special acknowledgment (ACK) frame to the sender indicating the correct receipt of a frame.
- The receiver also returns a negative acknowledgment (NACK) for incorrectly-received frames.
- This (NACK) is only a hint to the sender so that it can retransmit a frame right away without waiting for a timer to expire.

Timers:

- One problem that simple ACK/NACK schemes fail to address is recovering from a frame that is lost, and as a result, fails to seek an ACK or NACK.
- What happens if an ACK or NACK becomes lost?
- Retransmission timers are used to resend frames that don't produce an ACK. When sending a frame, schedule a timer to expire at some time after the ACK should have been returned. If the timer goes off, retransmit the frame.

Sequence Numbers:

- Retransmissions introduce the possibility of duplicate frames.
- To remove duplicates, add sequence numbers to each frame, so that a receiver can distinguish between new frames and repeats of old frames.

4. FLOW CONTROL

Flow control deals with throttling the speed of the sender to match that of the receiver. There are two approaches use to control the transmission rate of sender.

Feedback-based flow:

The receiver sends back information to the sender giving it permission to send more data or at least telling the sender how the receiver is doing.

Rate-based flow control:

The protocol has a built-in mechanism that limits the rate at which sender may transmit data, without using feedback from receiver.

Error Detection & Correction Control

In data communication, Error may occur due to signal attenuation, natural phenomenon such as lightning, which will affect a set of bits for a short time.

Detecting and correcting errors requires redundancy - sending additional information along with the data.

- **Error Detection**

Error detection is performed by calculating an error-detection code that is a function of bits being transmitted. The code is appended to the transmitted bits. The receiver calculates the code based on the incoming bits and compare it to the incoming code to check for errors.

- **Error Correction**

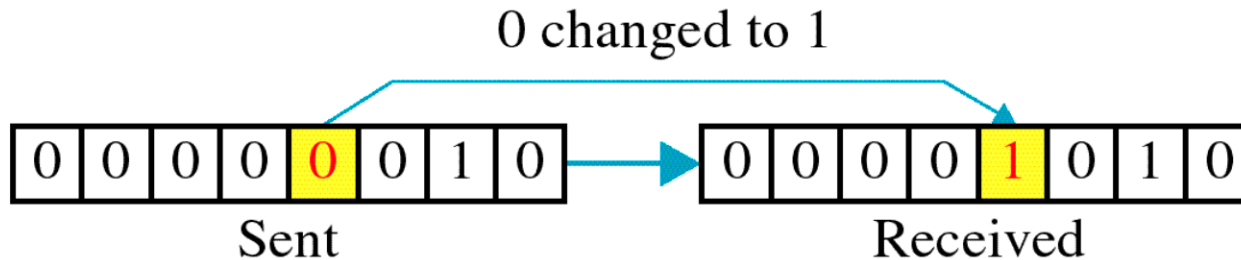
operates in a fashion similar to error detection but is capable of correcting certain errors in a transmitted bit stream.

- **Types of Errors**

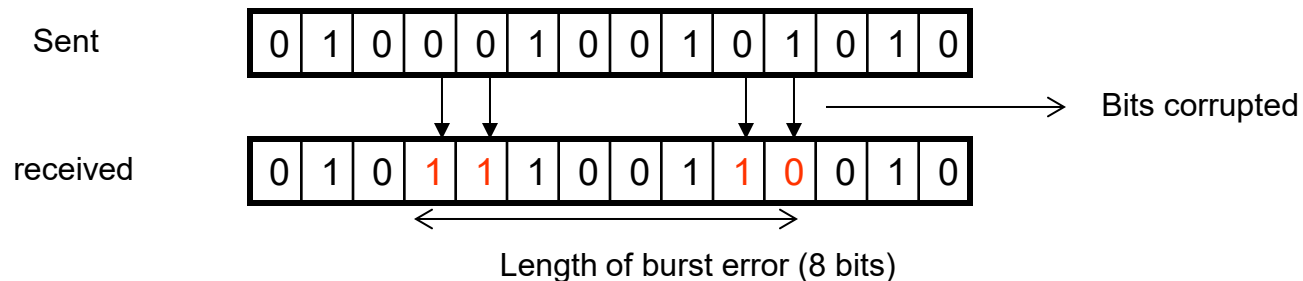
single bit error, burst error

There are two type of errors may occur during the transmission. Single bit error and burst error.

Single Bit Error: Only one bit in the data unit has changed.

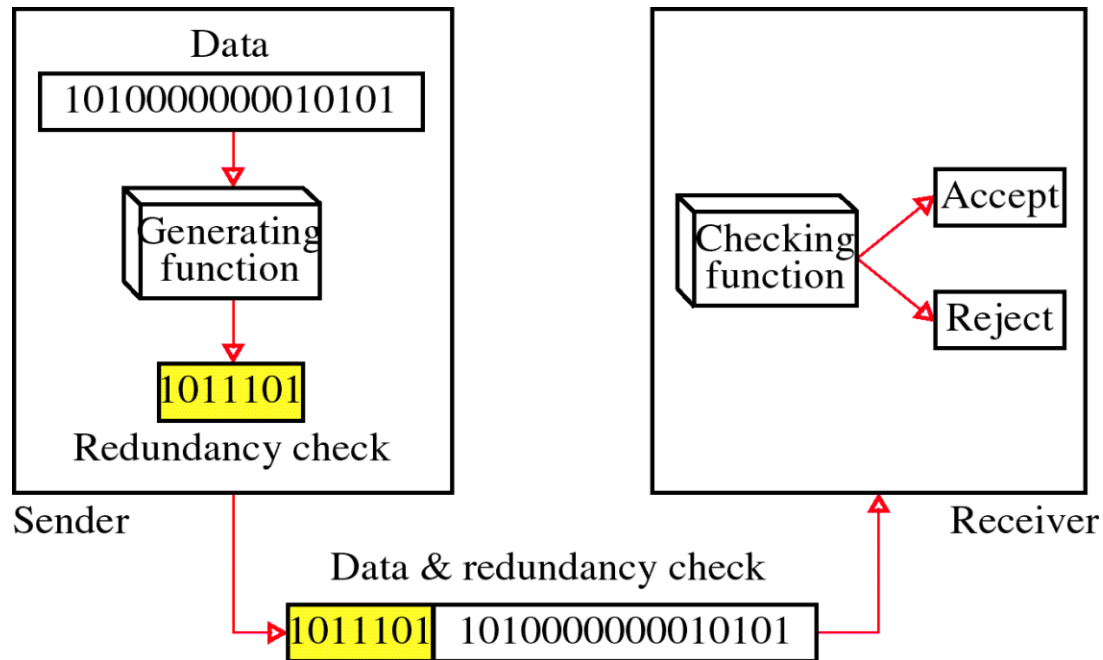


Burst Error: Two or more bits in the data unit have changed. The length of the burst is measured from the first corrupted bit to the last corrupted bit.



Redundancy

redundancy means adding extra bits to data unit for detecting errors at the receiver.

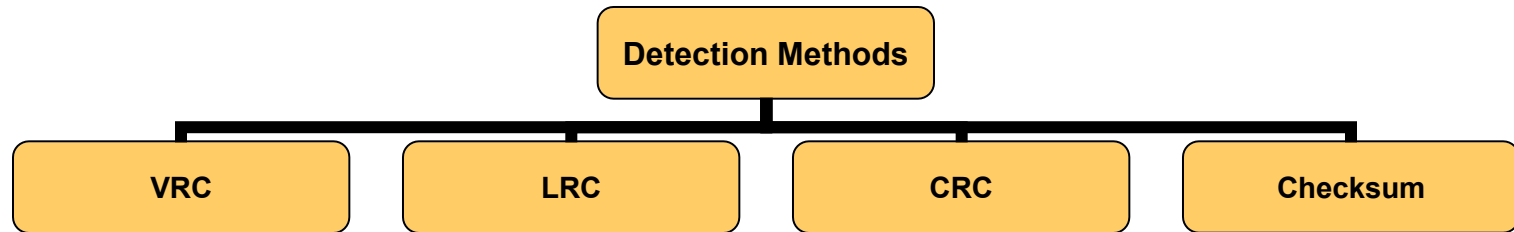


There are two ways of handling transmission errors:

Error Detection and correction: Include enough redundancy to detect and correct errors.

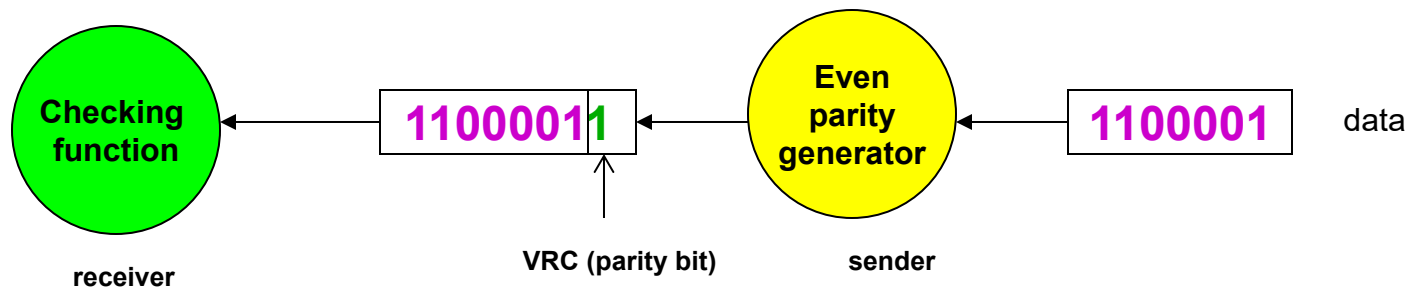
Error Detection and retransmission: Include enough redundancy bits to detect errors and use ACKs and retransmissions to recover from the errors.

Error Detection



Vertical Redundancy Check: (Parity Check):

A parity bit is added to every data unit so that the total number of 1s becomes even.



VRC can detect all single bit errors. It can detect burst errors only if the total number of errors in each data unit is odd.

Longitudinal Redundancy Check (LRC)

A block of bits is arranged in the matrix form and redundancy check (parity) is computed column wise. A redundant row of bits is added to the whole block.

Data: 11100111 11011101 00111001 10101001

LRC



1	1	1	0	0	1	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	0	1
1	0	1	0	1	0	0	1
1	0	1	0	1	0	1	0

VRC (parity row)

Data + LRC

11100111 11011101 00111001 10101001 10101010

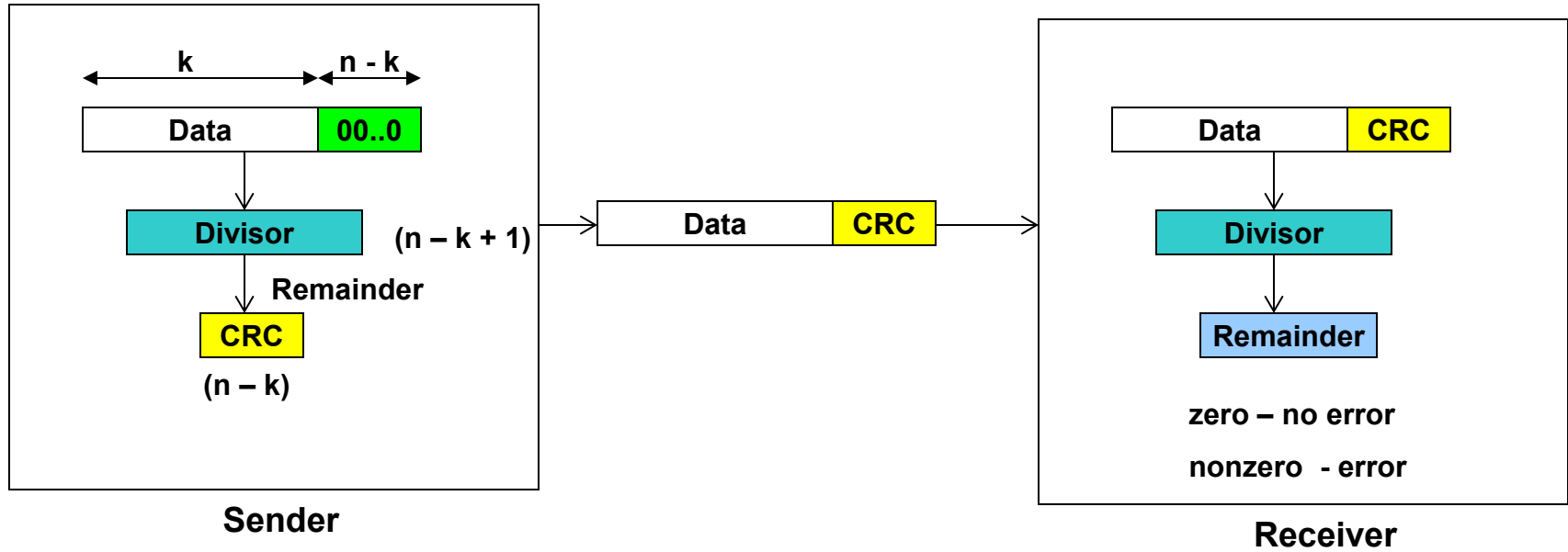


Direction of movement

A LRC of n bits can easily detect a burst error of n bits.

Cyclic Redundancy Check (CRC): Modulo 2 Arithmetic

Given a k -bit block of bits or message, the sender generate an $n - k$ bit sequence, known as a **CRC**, such that the resulting frame consisting of n bits, is exactly divisible by some predetermined number (**divisor**) of length $n - k + 1$. The receiver then divides the incoming frame by that number and, if there is no remainder, assumes there is no error.



T = n -bit frame to be transmitted

D = k -bit block of data or message

R = $(n - k)$ bits CRC

P = pattern of $n - k + 1$ bits; that is the predetermined divisor. The pattern p is chosen to be one bit longer than the CRC. The higher and lower order bits of P must be 1

Mathematically, It can describes as using modulo 2 arithmetic:

$$\frac{2^{n-k} D}{P} = Q + \frac{R}{P}$$

$$T = 2^{n-k} D + R$$

The errors in an n-bit frame can be presented by an n-bit filled with 1s in each error position.

$$T_r = T \oplus E \quad \oplus - XOR$$

T = Transmitted frame

E = Error pattern with 1s in positions where errors occur.

T_r = Received frame

If there is an error ($E \neq 0$), the receiver will fail to detect the error if and only if T_r is divisible by P , which is equivalent to E divisible by P

Modulo 2 Arithmetic:

Module 2 arithmetic uses binary addition with no carries (XOR), and binary subtraction with no carries also interpreted as the (XOR) operation.

$$\begin{array}{r} 1111 \\ +1010 \\ \hline 0101 \end{array}$$

$$\begin{array}{r} 1111 \\ -0101 \\ \hline 1010 \end{array}$$

Example of CRC using Modulo 2 Arithmetic

Message D = 1010001101

Pattern P = 110101

CRC R = to be calculated

Thus $n = 15$, $k = 10$, and $(n - k) = 5$

The message is multiply by 2^5 :

$2^5 D = 101000110100000$

This product is divided by P,
gives the remainder $R = 01110$

The remainder is added to $2^5 D$ ($T = 2^5 D + R$)
to give

$T = 101000110101110$, which is transmitted.

$$\begin{array}{r}
 \begin{array}{l} P \rightarrow 110101 \end{array} \overline{) \begin{array}{l} 1101010110 \\ 101000110100000 \end{array}} \quad \leftarrow 2^{n-k} D \\
 \underline{110101} \\
 111011 \\
 \underline{110101} \\
 111010 \\
 \underline{110101} \\
 111110 \\
 \underline{110101} \\
 101100 \\
 \underline{110101} \\
 110010 \\
 \underline{110101} \\
 01110 \quad \leftarrow \text{CRC}
 \end{array}$$

Sender

At the receiver

If receiver receives the frame T, which is divided by P

$$\begin{array}{r} 1101010110 \\ P \rightarrow 110101 \overline{) 101000110101110} \quad \leftarrow T \\ \underline{110101} \\ 111011 \\ \underline{110101} \\ 111010 \\ \underline{110101} \\ 111110 \\ \underline{110101} \\ 101111 \\ \underline{110101} \\ 110101 \\ \underline{110101} \\ 0 \leftarrow \text{remainder} \end{array}$$

There is no remainder, it is assumed that there is no errors.

Example

In a CRC detection scheme, choose $P = 10011$, message $M = 10010011011$. find the CRC and frame T .

- Suppose the channel introduces an error pattern 100010000000000 . (a flip from 1 to 0 or 0 to 1 in position 1 and 5). What is received. Can the error be detected?
- Repeat the part a with error pattern 100110000000000 .

Sol.

After calculation, CRC bits are 1100 . The frame $T = 100100110111100$ is sent.

a. $Tr = E \oplus T$

$$Tr = \begin{array}{r} 100010000000000 \\ 100100110111100 \\ \hline \end{array}$$

Frame 000110110111100 is received. Tr is divided by P ,

The remainder after division by P is 1110 , which is nonzero. The errors are detected.

b. $Tr = E \oplus T$

$$Tr = \begin{array}{r} 100110000000000 \\ 100100110111100 \\ \hline \end{array}$$

The string 000010110111100 is received, The remainder after division by P is zero. The errors are not detected

Cyclic Redundancy Check (CRC): Polynomial Method

All values of CRC process can be express as polynomials in a dummy variable X, with binary coefficients. Thus for ,

$$D = 110011, \text{ we have } D(X) = X^5 + X^4 + X + 1$$

$$P = 11001, \quad P(X) = X^4 + X^3 + 1$$

Arithmetic operations are modulo 2. The CRC process can now be described as:

$$\frac{X^{n-k} D(X)}{P(X)} = Q(X) + \frac{R(X)}{P(X)}$$

$$T(X) = X^{n-k} D(X) + R(X)$$

where $D \rightarrow \text{Data}$, $P \rightarrow \text{generator polynomial}$

$R \rightarrow \text{remainder}$, $T \rightarrow \text{frame to be transmitted}$

n - no of bits in a frame to be transmitted

k - no of bits block of data or message

$(n - k)$ bits CRC

$(n-k+1)$ bits divisor (generator polynomial)

Example of CRC using polynomial division:

Data $D = 1010001101$, we have $D(X) = X^9 + X^7 + X^3 + X^2 + 1$

Generator polynomial $P = 110101$, $P(X) = X^5 + X^4 + X^2 + 1$

$$\begin{array}{r}
 X^5 + X^4 + X^2 + 1 \overline{) \begin{array}{l} X^{14} + \phantom{X^{13}} + \phantom{X^{12}} + \phantom{X^{11}} + \phantom{X^{10}} + + + + + + + + + + 1 \\ X^{14} + X^{13} + \phantom{X^{12}} + \phantom{X^{11}} + X^{11} + X^9 \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + X^{12} + X^{11} + X^9 + X^8 \\ X^{13} + X^{12} + \phantom{X^{11}} + X^{10} + + \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + \phantom{X^{12}} + X^{10} + X^9 + X^7 \\ X^{11} + X^{10} + + + X^8 + X^6 \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + \phantom{X^{12}} + \phantom{X^{11}} + X^9 + X^8 + X^7 + X^6 + X^5 \\ X^9 + X^8 + + + X^6 + X^4 \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + \phantom{X^{12}} + \phantom{X^{11}} + \phantom{X^{10}} + X^7 + X^5 + X^4 \\ X^7 + X^6 + X^4 + X^2 \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + \phantom{X^{12}} + \phantom{X^{11}} + \phantom{X^{10}} + + X^6 + X^5 + + X^2 \\ X^6 + X^5 + X^3 + X \\ \hline \phantom{X^{14}} + \phantom{X^{13}} + \phantom{X^{12}} + \phantom{X^{11}} + \phantom{X^{10}} + + + X^3 + X^2 + X \end{array} } \leftarrow X^5 D(X)
 \end{array}$$

The resulting frame $T(X) = X^{14} + X^{12} + X^8 + X^7 + X^5 + X^3 + X^2 + X$

Checksum

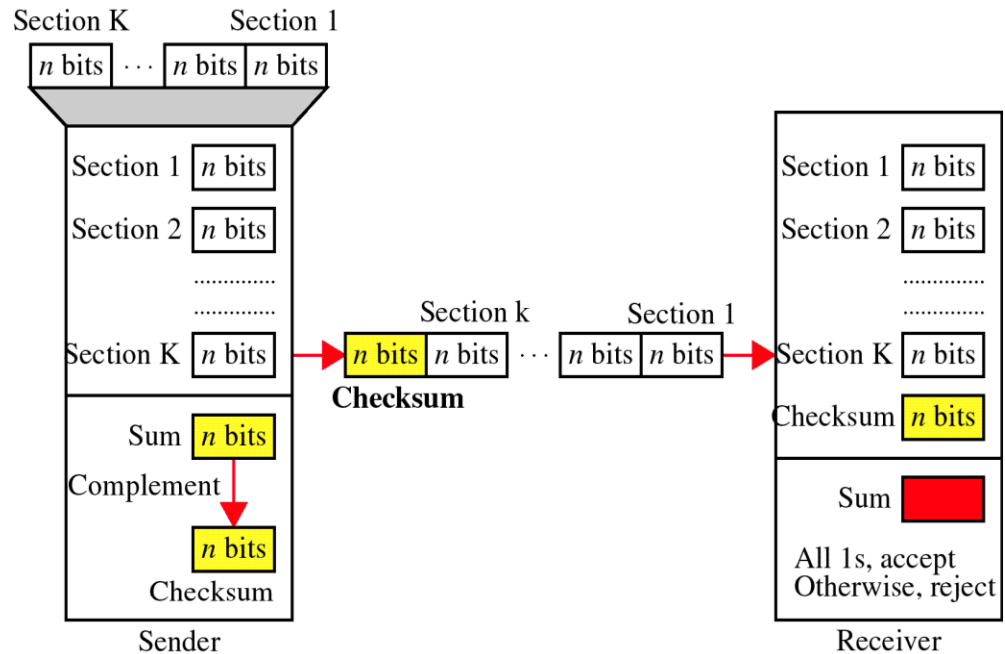
This error detection method is used by higher layer protocol.

The sender follows these step:

- The data unit is divided into k sections, each of n bits.
- All sections are added together using one's complement to the sum.
- The sum is complemented and become the checksum.
- The checksum is sent with the data.

The receiver follows these step:

- The data unit is divided into k sections, each of n bits.
- All sections are added together using one's complement to the sum.
- The sum is complemented.
- If the result is zero, the data are accepted: otherwise, they are rejected.



Example of checksum

Suppose the following block of 24 bits is to be sent using a checksum of 8 bits.

10101011 00101101 10010101, what is the resulting frame to be transmitted?

The data unit is divided into 3 section of each 8 bits.
And added using one's complement arithmetic.

10101011	
00101101	
10010101	
<u>01101101</u>	
_____	1 <i>carry</i>
01101110	<i>sum</i>
10010001	<i>checksum (complement)</i>

The frame sent is

← 10101011 00101101 10010101 10010001

Sender

Receiver receives the frame:

10101011 00101101 10010101 10010001

The pattern is divided into 4 section of each 8 bits
and added using one's complement arithmetic.

10101011	
00101101	
10010101	
<u>10010001</u>	
11111110	<i>sum</i>
_____	1 <i>carry</i>
11111111	<i>sum</i>
00000000	<i>complement</i>

zero means that there is no error.

Receiver

Error Correction Codes

Relationship between data bits and redundancy bits to correct single bit error:

A frame consists of m data bits and r redundant or check bits. Let the total length of frame be n (i.e. $n = m + r$). A n -bit unit containing data and check bit is often referred to as an n -bit codeword.



Number of states of single bit error = $m + r$

Number of state for no error = 1

So, number of redundant bit (r) that represent all states $m+r+1$ must satisfy

$$2^r \geq m + r + 1$$

Hamming code

This method uses extra parity bits to allow the identification of a single bit error. Create the codeword as follows:

1. Mark all bit positions that are powers of two as parity bits (1, 2, 4, 8, 16, 32, 64, 128, etc).
2. All other bit positions are for the data to be encoded (3, 5, 6, 7, 9, 10, 11, etc).
3. Each parity bit calculates the parity for some of the bits in the codeword. The position of the parity bit determines the sequence of bits that it alternately checks and skips.

Position 1: ckeck1 bit, skip 1 bit, check 1 bit, skip 1 bit, ...
 (1, 3, 5, 7, 9, 11, ...)

Position 2: check 2 bit, skip 2 bit, check 2 bit, skip 2 bit,...
 (2, 3, 6, 7, 10, 11, 14, 15, ...)

Position 4: check 4 bit, skip 4 bit, check 4 bit, skip 4 bit,...
 (4,5,6,7, 12,13,14,15, 20,21,22,24, ...)

position 8: check 8 bit, skip 8 bit, check 8 bit, skip 8 bit,...
 (8,9,10,11,12,13,14,15, 24,25,26,27,28,29,30,31, ...)

.....

4. Set the parity bit 1 if the total number of 1s in the potions it check is odd.
set the parity bit 0 if the total number of 1s in the potions it check is even.

Example: Hamming codeword

To construct the even parity hamming codeword for a data byte : 10011010.

1. To create the codeword, leaving the spaces for the parity bits.

 1 0 0 1 1 0 1 0

Position 1: check data bits 1, 3, 5, 7, 9, 11.

? 1 0 0 1 1 0 1 0 even parity, set bit position 1 to 0.

0 1 0 0 1 1 0 1 0

Position 2: check bit 2, 3, 6, 7, 10, 11.

0 ? 1 0 0 1 1 0 1 0 odd parity, set bit position 2 to 1.

0 1 1 0 0 1 1 0 1 0

Position 4: check bit, 4,5,6,7.

0 1 1 ? 0 0 1 1 0 1 0 odd parity, set bit position 4 to 1.

0 1 1 1 0 0 1 1 0 1 0

position 8: check bit 8,9,10,11,12

0 1 1 1 0 0 1 ? 1 0 1 0 even parity, set bit position 8 to 0

0 1 1 1 0 0 1 0 1 0 1 0

Codeword : 0 1 1 1 0 0 1 0 1 0 1 0

Finding and Fixing a bad bit in Hamming codeword

Check each parity bit, and add the positions that are wrong, this will give you the location of the bad bit.

Example:

The last example created a codeword 011100101010.

Suppose the codeword was received as 011100101110.

Verify each check bits, you will discover that parity bits 2 and 8 are incorrect. so,
 $2 + 8 = 10$.

10th position is the location of the bad bit. So correct codeword is : 011100101010